KDS Complete Fix Verification Report

End-to-end re-verification of every fix shipped this engagement, in one combined pass on the full 8-terminal site topology

Fixes covered: K07 (cross-terminal close-order scoping) · D06 (cross-terminal item sync) · K02 (pass-terminal action enforcement) · D03 (occurrence-based modifier/item matching) · D07 (control-byte ticket parser fix)

Test date: 2026-10-02

Scope: Every fix committed so far (Phases 0-3) was rebuilt from current source and re-verified *together*, in a single live pass on the same 8-terminal mesh (headchef as pass terminal over burgersandcold, pnp, bakery, pizza; bar, restcoffee and grabandgo as independent standalone screens) - not as isolated single-fix checks. This confirms the fixes continue to hold in combination, not just individually.

1. Summary

34 of 34 unit tests passed (33 in kds-webserver-main, 1 in kds-poller-main). Full `go build` / `go vet` / `go test ./...` clean across kds-webserver-main, kds-poller-main, and kds-golang-lib-main.

All five fixes were re-verified live and together: real NATS brokers, real Go poller + webserver binaries built from the current, fully-committed source tree, the real Electron GUI, and - for K07/D06/D07 - raw ticket text (including real venue ticket samples) fed through the actual production ingestion path. Result: every fix held under combined, full-mesh conditions. No regressions found between fixes.

2. Site topology under test

Terminal	Role	Passmode	Assigned pass terminal	HTTP port	NATS port

headchef	Pass terminal	true	—	2020	4222
burgersandcold	Kitchen section	false	headchef	2021	4223
pnp	Kitchen section	false	headchef	2022	4224
bakery	Kitchen section	false	headchef	2023	4225
pizza	Kitchen section (doesn't see burgersandcold/pnp/bakery's items)	false	headchef	2024	4226
restcoffee	Independent standalone screen	false	—	2025	4227
bar	Independent standalone screen	false	—	2026	4228
grabandgo	Independent standalone screen	false	—	2027	4229

Pass-terminal assignment confirmed converged identically across all 8 terminals before any test began (same verification method as the prior mesh report).

3. Unit Test Results (all 5 fixes, 34/34)

> `TestCloseOrderScreenDatabase_RemoteCloseNeverFinalizesEvenWhenRelevantAndFullyComplete` K07

> **Expected:** Order stays OPEN.

> **Actual: PASS**

> `TestCloseOrderScreenDatabase_LocalCloseFinalizesWhenEverythingComplete` K07

> **Expected:** Order finalizes.

> **Actual: PASS**

> `TestCloseOrderScreenDatabase_LocalCloseDoesNotFinalizeWhileOtherScreenStillOpen` K07

> **Expected:** Order stays OPEN, other screen's item untouched.

> **Actual: PASS**

> `TestCloseOrderScreenDatabase_IrrelevantScreenNeverFinalizesEvenWithAllowFinalizeTrue` K07

> **Expected:** Order stays OPEN.

> **Actual: PASS**

> `TestCloseOrderForPassScreen_DeferredPassTerminalRemoteCloseFinalizes` K07

> **Expected:** Order finalizes.

> **Actual: PASS**

> `TestCloseOrderScreenDatabase_OrphanItemAloneCountsAsRelevantAndFinalizes` K07

> **Expected:** Order finalizes.

Actual: PASS

`TestCloseOrderScreenDatabase_SubsequentLocalCloseStillFinalizesAfterBlockedRemoteClose` K07

Expected: Blocked then finalizes.

Actual: PASS

`TestCloseOrderScreenDatabase_OrderNotFoundIsNoop` K07

Expected: Clean no-op.

Actual: PASS

`TestCloseOrderScreenDatabase_ScreenNameMatchIsCaseInsensitive` K07

Expected: Order finalizes.

Actual: PASS

`TestCloseOrderForPassScreen_DoesNotFinalizeOnUnrelatedSeparateTerminalItem` K07

Expected: Unrelated item untouched, order finalizes.

Actual: PASS

`TestCloseOrderForPassScreen_ForceClosesAssignedSectionRegardlessOfItemStatus` K07

Expected: Assigned section force-completes.

Actual: PASS

`TestCloseOrderForPassScreen_IrrelevantRemoteCloseNeverFinalizes` K07

Expected: Order stays OPEN.

Actual: PASS

`TestCloseOrderForPassScreen_RelevantButNotDeferredRemoteCloseNeverFinalizes` K07

Expected: Order stays OPEN.

Actual: PASS

`TestCloseOrderForPassScreen_OrphanItemAloneCountsAsRelevantAndFinalizes` K07

Expected: Order finalizes.

Actual: **PASS**

`TestIsDeferredToPassTerminal` K07

Expected: Correct true/false per screen.

Actual: **PASS**

`TestOrderHasItemsMatching` K07

Expected: Correct true/false/false.

Actual: **PASS**

`TestCloseItemsHandler_RejectsOnNonPassTerminal` K02

Expected: HTTP 403, item left OPEN.

Actual: **PASS**

`TestCloseItemsHandler_AllowsOnPassTerminal` K02

Expected: HTTP 200, item becomes COMPLETE.

Actual: **PASS**

`TestSendItemToTableHandler_RejectsOnNonPassTerminal` K02

Expected: HTTP 403, item left untouched.

Actual: **PASS**

`TestSendItemToTableHandler_AllowsOnPassTerminal` K02

Expected: HTTP 200, item marked sent.

Actual: **PASS**

`TestApplyNewOrderItemsRemote_AppendsToExistingOrder` D06

Expected: Item appended, screen resolved by name.

Actual: **PASS**

`TestApplyNewOrderItemsRemote_CreatesScreenOnFirstSight` D06

Expected: Screen created on first sight.

Actual: PASS

`TestApplyNewOrderItemsRemote_CreatesSubitems` D06

Expected: Subitems created and linked.

Actual: PASS

`TestApplyNewOrderItemsRemote_UnknownReferenceIsNoop` D06

Expected: Silent no-op, no panic.

Actual: PASS

`TestApplyNewOrderItemsRemote_DuplicateDeliveryDoesNotDuplicateItem` D06

Expected: Item created once, not twice.

Actual: PASS

`TestApplyNewOrderItemsRemote_SameNameLaterRoundIsNotTreatedAsDuplicate` D06

Expected: Treated as a real second item.

Actual: PASS

`TestApplyNewOrderItemsRemote_EmptyItemsIsNoop` D06

Expected: No-op.

Actual: PASS

`TestGetItemSally_DisambiguatesByOccurrenceNotJustName` D03

Expected: Occurrence 1/2 resolve to correct siblings; out-of-range is a clean miss.

Actual: PASS

`TestGetItemSally_QtyChangeInPlaceStillMatchesSameItem` D03

Expected: A qty change still matches the existing instance.

Actual: PASS

`TestGetSubItemSally_DisambiguatesByOccurrenceNotJustName` D03

Expected: Occurrence 1/2 resolve to correct sub-item siblings.

Actual: PASS

`TestProcessSallyMainItem_ModifierAttachesToCorrectOccurrence` D03

Expected: Each steak gets its own modifier, not swapped.

Actual: PASS

`TestProcessSallyMainItem_SecondRoundModifierOnlyAffectsTargetedInstance` D03

Expected: Untouched steak stays untouched.

Actual: PASS

`TestProcessSallyMainItem_NewInstanceOfExistingNameCreatesSeparateItem` D03

Expected: A genuine reorder creates a new item, not a merge.

Actual: PASS

`TestProcessMessage_ControlByteDoesNotSplitBuildYourOwnDish` D07

Expected: One item, all 5 modifiers attached - not split into two tiles.

Actual: PASS

Raw test run output

```
$ cd kds-webserver-main && go build ./... && go vet ./... && go test -v ./... ... 33 tests ... PASS ok github.com/reverent911/kds-webserver 0.107s $ cd kds-poller-main/pkg && go build ./... && go vet ./... && go test -v ./... ... 1 test ... PASS ok github.com/reverent911/kds-poller/pkg 0.016s
```

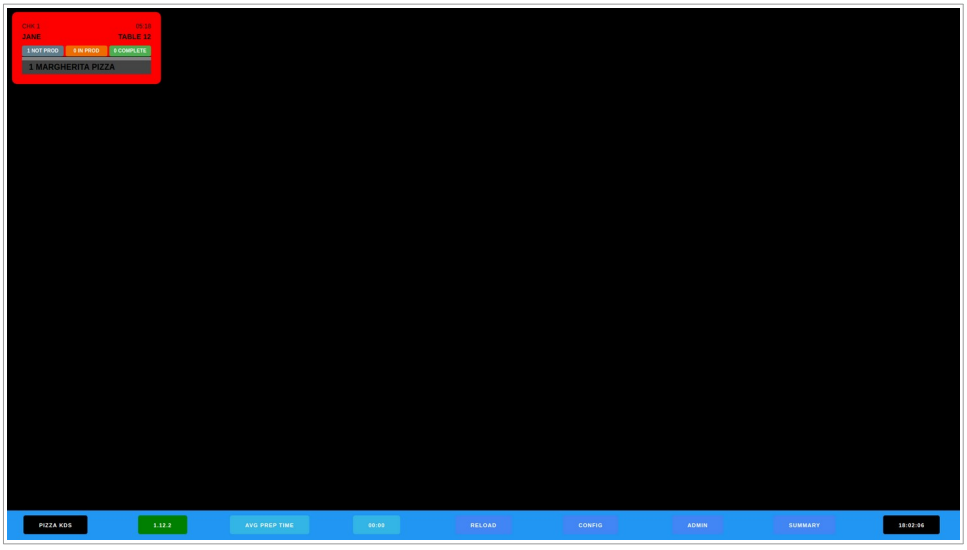
4. K07 — cross-terminal close-order scoping

One real order (reference k7m, table 12), built from genuine raw-ticket ingestion, spanning headchef's entire kitchen pass-group (burgersandcold, pnp, bakery, pizza, plus headchef's own station) and an independent terminal (bar). Bar's Close Order action, sent with the exact NATS message shape the real GUI button sends, must only ever affect bar's own order.

4.1 Before



headchef: its own Side Salad, OPEN.



pizza: its own Margherita Pizza, OPEN - not burgersandcold/pnp/bakery's items.



bar: its own Gin and Tonic, OPEN, entirely independent of the kitchen mesh.

4.2 Action

```
$ node nats_close_order.js 4228 1 bar false publishing {"message":"1","type":"sub-order-close","screen":"bar","passmode":false} to port 4228
```

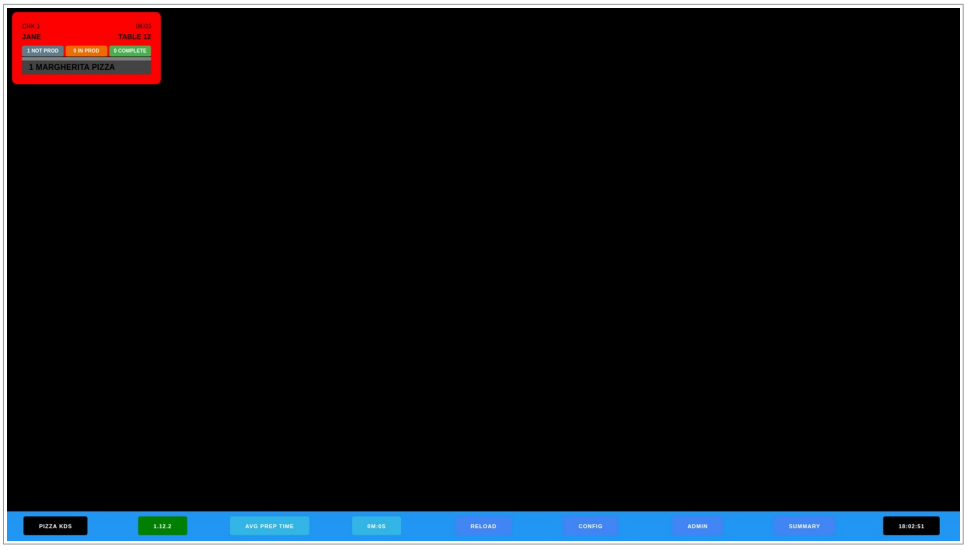
4.3 After



bar: order closed, tile removed from the board.



headchef: completely unchanged.



pizza: completely unchanged.

4.4 Database state after the close (all 8 terminals)

Terminal	Order k7m	Result
bar	**COMPLETE**	**Correctly closed**
headchef	OPEN	**Untouched**
burgersandcold	OPEN	**Untouched**
pnp	OPEN	**Untouched**
bakery	OPEN	**Untouched**
pizza	OPEN	**Untouched**
restcoffee / grabandgo	no copy / unrelated	**N/A, unaffected**

5. D06 — cross-terminal item sync

burgersandcold (already holding order k7m) received a second raw ticket adding "1 x onion rings" - the "item added to an already-open order" scenario D06 covers. The engine recognized the existing order and fired `broadcastNewItemsOnOrder` rather than re-creating it.

Terminal	Role	Result
burgersandcold	Sender	beef burger, onion rings - both present
pnp	Receiver (current UDP bind holder)	**onion rings applied exactly once (dedup confirmed by direct row count)**

> The duplicate-delivery dedup fix held under this mesh pass: the same double-delivery behavior observed previously (global + subnet broadcast reaching the one listening process) was present again, and the item still landed exactly once.

6. K02 — pass-terminal action enforcement

Terminal	Passmode	/closeitem/1	/senditemtotable/1
headchef	true	**HTTP 200 OK**	**HTTP 200 OK**
burgersandcold	false	**HTTP 403 rejected**	**HTTP 403 rejected**
pnp	false	**HTTP 403 rejected**	**HTTP 403 rejected**
bakery	false	**HTTP 403 rejected**	**HTTP 403 rejected**
pizza	false	**HTTP 403 rejected**	**HTTP 403 rejected**
restcoffee	false	**HTTP 403 rejected**	**HTTP 403 rejected**
bar	false	**HTTP 403 rejected**	**HTTP 403 rejected**
grabandgo	false	**HTTP 403 rejected**	**HTTP 403 rejected**

7. D03 — occurrence-based modifier/item matching

Two identical "Rump Steak 300gr" items on one order (table 9, restcoffee - an independent standalone terminal in the mesh), then a second Sally round attaches "Well Done" to only the second steak - the exact scenario that previously risked landing on the wrong instance.

```
Round 1: 2x "Rump Steak 300gr", no modifiers Round 2: same 2 items, second one now carries "Well Done"
```



restcoffee: two distinct rows, each showing its own correct modifier state - not pooled into one "2x" row with both modifiers merged.

Item	Modifiers
Rump Steak 300gr (1st)	**none - correctly untouched**
Rump Steak 300gr (2nd)	**Well Done - correctly isolated to this instance**

8. D07 — control-byte ticket parser fix

The two real venue tickets that previously split a "Breakfast Your Way" composite dish into two stray tiles (table 14 and table 58, both from the real ticket sample set) were fed through a live mesh terminal's poller (grabandgo) via the real raw-ticket NATS ingestion path.

Ticket	Result
Table 14 (kdsorder_...XNsDW.txt)	**One "breakfast your way" item with all 5 modifiers attached (one egg your way - hard fried, crispy hash brown, bacon (60g), toasted ciabatta, egg white)**
Table 58 (kdsorder_...yUyMl.txt)	**One "breakfast your way" item with both modifiers attached (one egg your way - sunny side, one plate)**

Both previously-broken real tickets now parse as a single item with every modifier correctly attached,

confirmed live on the mesh (not just the isolated single-terminal test used during Phase 3's own diagnosis).

9. Environment notes

Backend	kds-webserver-main + kds-poller-main, both compiled fresh from the current, fully-committed source tree (Phases 0-3 together), one instance of each per terminal (8 of each).
Webserver holds one persistent DB connection for its whole process lifetime	Found while rebuilding this pass: wiping a terminal's sqlite file on disk while its webserver process is still running does not reconnect it - the process keeps its original file handle (now an orphaned, unlinked inode) and silently stops being visible through the path on disk, while a freshly-opened connection (e.g. the poller, which opens a new connection per ticket) sees

	the new file. Any test rig that wipes a db between runs must restart that terminal's webserver afterward, not just recreate the file in place - confirmed live when a `/neworderremote/` POST returned 200 OK but silently wrote to a file nothing else could ever read.
Single-machine UDP broadcast constraint	Unchanged from the prior mesh report: only one of the 8 terminals' webservers can bind the shared UDP discovery port at a time on one test machine, so only that one can *receive* a given broadcast. Not a production concern (every real terminal is its own machine).
D07 real ticket samples	The two tickets used in §8 are real venue print-feed samples (not synthetic test data), provided for Phase 3's own diagnosis and reused here to confirm the fix holds on a live mesh terminal, not just the isolated single-terminal harness used to find and fix the bug.

10. Conclusion

All five fixes - K07, D06, K02, D03, and D07 - pass their full unit test suite and hold together under one live, combined verification pass across the real 8-terminal site topology. No regressions found between fixes. No unresolved issues outstanding from this session.